

TECNICATURA UNIVERSITARIA EN PROGRAMACION A DISTANCIA

Programación 1

Trabajo Practico Integrador

Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Institucion	Universidad Tecnologica Nacional
Carrera	Tecnicatura Universitaria en programación a Distancia
Materia	Programacion 1
Estudiante 1	Tomas Lopez Turconi (Comisión: M26 C1-25)
Estudiante 2	Marcos Borquez (Comisión: M26 C1-17)
Fecha de entrega	Completar fecha

Indice

1. Introduccion	4
2. Objetivo del sistema	4
3. Marco teorico	4
3.1 Listas	4
3.2 Diccionarios	4
3.3 Funciones	4
3.4 Condicionales	4
3.5 Estructuras repetitivas	4
3.6 Ordenamientos	4
3.7 Estadisticas basicas	5
3.8 Archivos CSV	5
4. Decisiones tecnicas y arquitectura	5
4.1 Estructura de un registro	5
4.2 Diagrama de flujo	5
5. Funcionalidades implementadas	6
5.1 Agregar pais	6
5.2 Actualizar datos	6
5.3 Busqueda por nombre	6
5.4 Filtros	6
5.5 Ordenamientos	6
5.6 Estadisticas	6
6. Validaciones y manejo de errores	6
7. Capturas de ejecucion	6
8. Dificultades y resoluciones	8
9. Conclusiones	8
10. Bibliografia y webgrafia	8

11. Links del proyecto	8
------------------------------	---

1. Introducción

El presente trabajo practico integrador desarrolla una aplicación de consola en Python para gestionar información de países. El sistema permite leer datos desde un archivo CSV, almacenarlos en una lista de diccionarios, realizar búsquedas, aplicar filtros, ordenar registros y calcular estadísticas básicas sobre el dataset.

El objetivo principal es integrar los contenidos trabajados durante Programacion 1: estructuras de datos, modularización mediante funciones, condicionales, bucles, lectura de archivos, validaciones y manejo básico de errores.

2. Objetivo del sistema

El sistema busca facilitar la gestión y consulta de datos de países. Cada país se representa con su nombre, población, superficie y continente. A partir de esa información, el usuario puede agregar nuevos países, actualizar datos existentes, buscar países por nombre, filtrar por diferentes criterios, ordenar la información y obtener indicadores estadísticos.

3. Marco teórico

3.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable. En este proyecto se utiliza una lista para almacenar todos los países cargados desde el archivo CSV. Cada posición de la lista contiene un diccionario que representa un país.

```
países = [  
    {"nombre": "Argentina", "población": 45376763, "superficie": 2780400,  
     "continente": "America"}  
]
```

3.2 Diccionarios

Un diccionario almacena datos mediante pares clave-valor. En el sistema, cada país se guarda como un diccionario con las claves nombre, población, superficie y continente. Esta estructura facilita acceder a los datos por su significado, por ejemplo `país['nombre']` o `país['población']`.

3.3 Funciones

Las funciones permiten dividir el programa en partes más pequeñas y reutilizables. En este proyecto cada función tiene una responsabilidad concreta, por ejemplo cargar el CSV, agregar un país, buscar, filtrar, ordenar o mostrar estadísticas. Esto mejora la organización del código y facilita su mantenimiento.

3.4 Condicionales

Las estructuras condicionales `if`, `elif` y `else` permiten tomar decisiones durante la ejecución. En el programa se usan para validar datos, seleccionar opciones del menú, determinar si una búsqueda tiene resultados y decidir qué función ejecutar según la opción ingresada por el usuario.

3.5 Estructuras repetitivas

Los bucles while y for permiten repetir instrucciones. El while se utiliza para mantener activo el menú principal hasta que el usuario decida salir. El for se utiliza para recorrer la lista de países al buscar, filtrar, mostrar datos o calcular estadísticas.

3.6 Ordenamientos

El ordenamiento consiste en reorganizar los datos según un criterio. En el sistema se utiliza sorted() para ordenar la lista de países por nombre, población o superficie, tanto de manera ascendente como descendente.

3.7 Estadísticas básicas

Las estadísticas básicas permiten obtener información resumida del dataset. En el proyecto se calcula el país con mayor y menor población, el promedio de población, el promedio de superficie y la cantidad de países por continente.

3.8 Archivos CSV

CSV significa valores separados por comas. Es un formato simple para almacenar datos tabulares. El proyecto lee los países desde países.csv usando el módulo csv de Python. También guarda los cambios realizados por el usuario en el mismo archivo.

4. Decisiones técnicas y arquitectura

El proyecto se organizó en un archivo principal llamado gestion_paises.py y un dataset base llamado países.csv. Se eligió una lista de diccionarios porque permite representar múltiples países y acceder a sus datos de manera clara mediante claves.

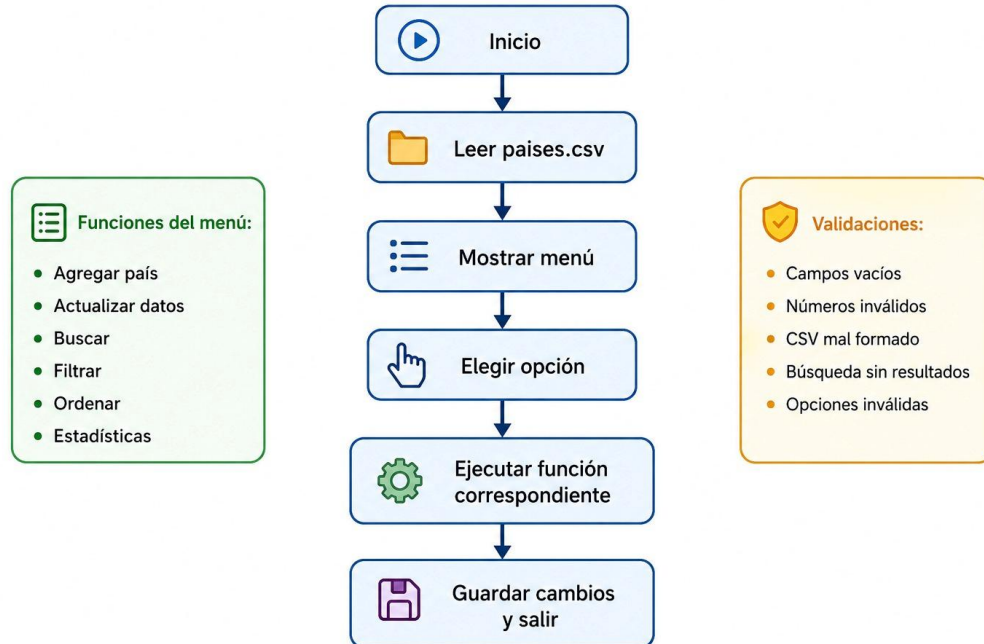
El sistema trabaja de forma modular: una función se encarga de leer el CSV, otra de guardar datos, otras de validar entradas, y cada funcionalidad del menú se resuelve en funciones independientes.

4.1 Estructura de un registro

```
{  
    "nombre": "Argentina",  
    "población": 45376763,  
    "superficie": 2780400,  
    "continente": "America"  
}
```

4.2 Diagrama de flujo

Flujo principal del sistema



5. Funcionalidades implementadas

El programa ofrece un menú interactivo de consola con las siguientes opciones: mostrar todos los países, agregar un país, actualizar población y superficie, buscar por nombre, filtrar países, ordenar países, mostrar estadísticas, guardar cambios y salir.

5.1 Agregar país

La opción de agregar país solicita nombre, población, superficie y continente. No permite campos vacíos, evita duplicados y valida que población y superficie sean números enteros válidos.

5.2 Actualizar datos

La actualización permite modificar población y superficie de un país existente. Si el país no se encuentra, se informa al usuario sin interrumpir el programa.

5.3 Búsqueda por nombre

La búsqueda permite coincidencia parcial o exacta. Por ejemplo, si se ingresa 'arg', el sistema puede encontrar Argentina.

5.4 Filtros

El sistema permite filtrar por continente, rango de población y rango de superficie. En todos los casos se validan entradas inválidas y se informa cuando no hay resultados.

5.5 Ordenamientos

Los datos se pueden ordenar por nombre, población o superficie. El usuario puede elegir si el orden será ascendente o descendente.

5.6 Estadísticas

Las estadísticas muestran indicadores clave del dataset: país con mayor población, país con menor población, promedio de población, promedio de superficie y cantidad de países por continente.

6. Validaciones y manejo de errores

El programa utiliza try/except para evitar fallos ante entradas invalidas. Por ejemplo, si el usuario escribe texto cuando se espera un numero entero, el sistema captura el error y muestra un mensaje claro. También se controlan errores de formato en el CSV, campos vacíos, valores negativos y búsquedas sin resultados.

Las validaciones principales son: campos no vacíos, población mayor o igual a cero, superficie mayor que cero, países duplicados, rangos inválidos y opciones fuera del menú.

7. Capturas de ejecución

7.1 Búsqueda por nombre

```
$ python gestion_paises.py

=====
GESTION DE DATOS DE PAISES
=====

1. Mostrar todos los paises
2. Agregar un pais
3. Actualizar poblacion y superficie
4. Buscar pais por nombre
5. Filtrar paises
6. Ordenar paises
7. Mostrar estadisticas
8. Guardar cambios
9. Salir

Seleccione una opcion: 4
Ingrese el nombre o parte del nombre a buscar: arg

Listado de paises:
Argentina          45376763          2780400 America
```

7.2 Filtro por continente

```

Seleccione una opcion: 5

Filtros disponibles:
1. Filtrar por continente
2. Filtrar por rango de poblacion
3. Filtrar por rango de superficie

Seleccione una opcion de filtro: 1
Ingrese el continente a filtrar: Asia

Listado de paises:
Japon          125800000      377975 Asia
China          1412000000     9596961 Asia
India          1408000000     3287263 Asia

```

7.3 Estadísticas

```

Seleccione una opcion: 7

Estadísticas del dataset
-----
Pais con mayor poblacion: China (1412000000)
Pais con menor poblacion: Nueva Zelanda (5123000)
Promedio de poblacion: 251759292.33
Promedio de superficie: 3158153.47 km2

Cantidad de paises por continente:
- America: 5
- Asia: 3
- Europa: 3
- Oceania: 2
- Africa: 2

```

8. Dificultades y resoluciones

Una dificultad del proyecto fue organizar el código para que cada función tuviera una responsabilidad concreta. Esto se resolvió separando la lógica en funciones específicas: lectura del CSV, validaciones, búsquedas, filtros, ordenamientos y estadísticas.

Otra dificultad fue controlar datos inválidos sin que el programa se interrumpiera. Para resolverlo se incorporó manejo de errores con try/except y mensajes claros para el usuario.

También se consideró el problema de los datos cargados desde el CSV. Si una fila tiene errores de formato, el sistema informa la fila inválida y continúa leyendo el resto del archivo.

9. Conclusiones

El trabajo permitió integrar los contenidos centrales de Programación 1 en un programa funcional. Se aplicaron listas, diccionarios, funciones, condicionales, bucles, lectura y escritura de archivos CSV, validaciones, filtros, ordenamientos y cálculo de estadísticas.

La modularización facilitó la comprensión del código y permitió que cada parte del sistema pudiera probarse y explicarse por separado. El uso de archivos CSV agrega una forma simple de persistir datos y trabajar con información externa al código fuente.

10. Bibliografía

Python Software Fundación. documentación oficial de Python 3: <https://docs.python.org/3/>

PythonSoftware Foundation. Modulo csv: <https://docs.python.org/3/library/csv.html>

Python Software Foundation. Estructuras de datos:

<https://docs.python.org/3/tutorial/datastructures.html>

Python Software Foundation. Funciones incorporadas: <https://docs.python.org/3/library/functions.html>

Real Python. Python CSV: <https://realpython.com/python-csv/>

11. Links del proyecto

[Link al repositorio de GitHub](#)

[Link al video demostrativo por Drive](#)